

Политика безопасности SQL Server

1. Область применения

Политика распространяется на все экземпляры SQL Server (разработка, тестирование, production) в организации.

2. Аутентификация

2.1. Режим аутентификации

Используется **только режим Windows Authentication** (Integrated Security) для доменных учётных записей, если нет технических ограничений.

SQL Server Authentication разрешается только для устаревших приложений, не поддерживающих интеграцию с Active Directory, с обязательным использованием сложных паролей и аудита.

2.2. Разделение «Имя входа» и «Пользователь базы данных»

Определения (согласно стандарту организации):

Имя входа (Login) – учётная запись уровня экземпляра, используемая для аутентификации на SQL Server.

Пользователь базы данных (Database User) – участник (security principal) внутри конкретной базы данных, связанный с именем входа.

Принципы:

Имя входа и связанный пользователь БД могут иметь разные имена (например, corp\petrov → пользователь petrov_db).

В production-средах имена должны различаться, чтобы избежать прямой привязки к доменному логину в коде приложения.

Для служебных учётных записей (приложения, службы) используется соглашение:

- Логин: app_billing_svc
- Пользователь БД: billing_user

2.3. Сопоставление логинов и пользователей

Каждое имя входа должно быть явно сопоставлено с пользователем в каждой базе данных, где требуется доступ.

Запрещено использовать гостевую учётную запись (**guest**) в базах данных, кроме специально разрешённых случаев.

3. Управление доступами

3.1. Принцип наименьших привилегий (PoLP)

Имя входа получает только права уровня сервера, необходимые для подключения (обычно **CONNECT SQL**, **VIEW ANY DATABASE** – только для администраторов).

Основные права назначаются на уровне базы данных через роли: db_datareader, db_datawriter, пользовательские роли.

3.2. Роли сервера

sysadmin – не более 2 выделенных учётных записей, использование через защищённое рабочее место (PAW).

securityadmin – только для группы управления доступом.

3.3. Владельцы баз данных

Избегать dbo, равного sysadmin.

Для каждой БД назначить выделенного владельца – отдельное имя входа без прав sysadmin.

4. Пароли и учётные записи SQL Server Authentication

Параметр	Требование
Длина пароля	≥ 12 символов
Сложность	буквы, цифры, спецсимволы
Срок действия	90 дней (принудительно через политику Windows, если используем Windows Auth, либо CHECK_EXPIRATION = ON для SQL логинов)
Проверка политики	CHECK_POLICY = ON (для SQL логинов)

5. Аудит и мониторинг

Обязательно логировать:

- Успешные и неудачные попытки входа (failed login, successful login).
- Изменения разрешений (GRANT, REVOKE, DENY).
- Создание/удаление логинов и пользователей.

Используется SQL Server Audit (на уровне экземпляра) или расширенные события.

6. Именованное и документация

Соглашение для production:

Имя входа для приложения: app_<система>_<среда> (например, app_crm_prod)

Пользователь в БД: app_<система>_user

Все сопоставления логин ↔ пользователь БД хранятся в репозитории CMDB или в защищённой Excel-таблице с контролем версий.

7. Шифрование

7.1. Шифрование данных при передаче (TLS/SSL)

Все соединения с SQL Server **должны использовать шифрование** (Force Encryption = Yes на стороне сервера либо Encrypt=true в строке подключения).

Используется доверенный сертификат, выданный корпоративным центром сертификации (не самоподписанный в production).

Протокол TLS версии **не ниже 1.2**.

7.2. Шифрование данных при хранении (TDE)

Прозрачное шифрование данных (TDE) включается для всех баз данных, содержащих:

- Персональные данные (GDPR, ФЗ-152)
- Финансовую или платёжную информацию (PCI DSS)
- Коммерческую тайну

TDE шифрует файлы данных (.mdf, .ndf) и журнал (.ldf) на уровне страниц.

Ключ шифрования базы данных (DEK) защищается сертификатом, хранящимся в **master** базе данных.

Сертификат TDE должен регулярно архивироваться и храниться отдельно от резервных копий пользовательских БД.

7.3. Шифрование столбцов (Always Encrypted)

Для особо чувствительных данных (пароли, номера карт, медицинские данные) применяется **Always Encrypted**.

Шифрование выполняется на стороне клиента (драйвером), сервер работает только с зашифрованными значениями.

Ключи шифрования столбцов (Column Encryption Key) хранятся в надёжном хранилище (Windows Certificate Store, Azure Key Vault, HSM).

7.4. Управление ключами

Все ключи и сертификаты имеют владельца, срок действия и процедуру ротации.

Резервные копия мастер-ключа (Service Master Key) и сертификатов TDE хранятся в защищённом хранилище (например, HashiCorp Vault).

8. Безопасность резервных копий (Backup security)

8.1. Шифрование резервных копий

Все резервные копии баз данных **обязательно шифруются** при создании.

Использовать:

```
BACKUP DATABASE [Sales]
TO DISK = '\\backup\Sales.bak'
WITH ENCRYPTION (
    ALGORITHM = AES_256,
    SERVER CERTIFICATE = [BackupCert]
```

);

Алгоритм шифрования — AES-256.

Сертификат для шифрования бэкапов отличается от сертификата TDE.

8.2. Хранение резервных копий

Бэкапы не хранятся на том же физическом диске, что и исходные данные.

Доступ к папкам с резервными копиями имеют только:

- Служба SQL Server (учётная запись запуска)
- Резервная команда (администраторы бэкапов)

Запрещены общедоступные сетевые шары с бэкапами.

8.3. Контроль целостности

Каждый бэкап проверяется командой `RESTORE VERIFYONLY`.

Периодически выполняются **тестовые восстановления** на изолированном сервере (не реже 1 раза в месяц для production БД).

Результаты проверок документируются.

8.4. Хранение и ротация

Схема: полная резервная копия (еженедельно) + разностные (daily) + журнала транзакций (каждые 15–60 минут для критических БД).

Хранение:

- Ежедневные бэкапы – 30 дней
- Еженедельные – 3 месяца
- Ежемесячные – 1 год

Старые бэкапы уничтожаются с подтверждением (акт уничтожения).

8.5. Резервное копирование системных БД

Регулярно бэкапить: `master`, `msdb`, `model` (а также `ssisdb` при наличии).

Резервная копия `master` восстанавливается на ту же версию SQL Server, что и оригинал.

9. Регулярные проверки

9.1. Периодичность проверок

Тип проверки	Периодичность	Исполнитель
Аудит учётных записей (логины, пользователи, роли)	Ежемесячно	DBA или Security team
Проверка неиспользуемых/лишних прав	Ежемесячно	Security team
Анализ неудачных попыток входа	Еженедельно	DBA

Проверка настроек шифрования и сертификатов	Ежеквартально	Security team
Тестовое восстановление из бэкапа	Ежемесячно	DBA
Сканирование уязвимостей (например, Microsoft SQL Vulnerability Assessment)	Ежеквартально	Security team
Обновление версии SQL Server (патчи, CU)	По графику (или внепланово для критических уязвимостей)	DBA + IT Ops

9.2. Содержание проверки учётных записей

Выявить имена входа с правом `sysadmin` – обосновать каждого.

Найти пользователей БД без привязки к активному имени входа (`ORPHANED USERS`):

EXEC sp_change_users_login @Action='Report';

Удалить учётные записи, не использовавшиеся более 90 дней.

9.3. Проверка разрешений

Выдать отчёт по `GRANT`, `DENY`, `REVOKE` на критических объектах (таблицы с ПИ, хранимые процедуры с повышением прав).

Для каждой нестандартной роли – описание и утверждённый запрос.

Устранить `db_owner` у прикладных пользователей, кроме случаев крайней необходимости.

9.4. Сканирование уязвимостей

Использовать **SQL Vulnerability Assessment** (встроенный в SSMS / Azure Data Studio).

Чек-лист обязательных проверок:

- Выявлены ли логины с пустыми или простыми паролями.
- Включён ли аудит.
- Не предоставлен ли доступ `public` к чувствительным объектам.
- Не используется ли устаревший протокол TLS (1.0/1.1).

9.5. Отчётность и реагирование

По итогам каждой проверки формируется **отчёт о соответствии политике безопасности**.

Критические нарушения (например, отсутствие шифрования бэкапов, учётная запись `sa` с известным паролем) исправляются **немедленно**, но не позже чем через 48 часов.

Некритические нарушения – план устранения с указанием сроков.

9.6. Документирование

Все проверки регистрируются в системе учёта.

История изменений прав, логинов, политик хранится не менее 3 лет (или дольше, если требуется регуляторами).

Пример создания зашифрованных резервных копий алгоритмом AES-256

SQL Server позволяет шифровать резервные копии с использованием встроенных алгоритмов (AES-128, AES-192, AES-256). Для этого применяется серверный сертификат, который хранится в базе master.

Шифрование резервных копий необходимо для:

- защиты данных при хранении;
- предотвращения несанкционированного доступа;
- соответствия требованиям информационной безопасности.

Подготовка среды

Создать папку для хранения:

- резервных копий
- сертификатов

C:\SQLBackups. Папка должна быть доступна службе SQL Server (права на запись)

Создание мастер-ключа. Создает и защищает сертификаты и ключи, хранится в базе master

```
USE master;  
GO
```

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'StrongPassword_12345!';  
GO
```

Создание сертификата. Используется для шифрования резервной копии, хранится в master

```
CREATE CERTIFICATE TDECert  
WITH SUBJECT = 'Certificate for backup encryption';  
GO
```

Проверка сертификата. Проверяется наличие сертификата и срок действия

```
SELECT name, subject, expiry_date  
FROM sys.certificates  
WHERE name = 'TDECert';  
GO
```

Резервное копирование сертификата. Происходит сохранение сертификата и ключа, необходимого для восстановления базы. Без сертификата восстановить зашифрованный backup невозможно

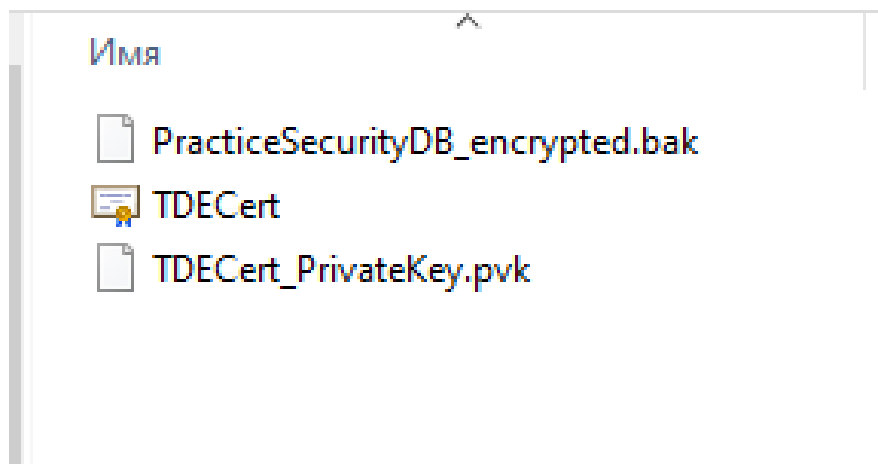
```
BACKUP CERTIFICATE TDECert  
TO FILE = 'C:\SQLBackups\TDECert.cer'  
WITH PRIVATE KEY (  
FILE = 'C:\SQLBackups\TDECert_PrivateKey.pvk',  
ENCRYPTION BY PASSWORD = 'StrongPrivateKeyPassword_123!'  
);  
GO
```

Создание зашифрованного архива

```
BACKUP DATABASE PracticeSecurityDB  
TO DISK = 'C:\SQLBackups\PracticeSecurityDB_encrypted.bak'  
WITH ENCRYPTION (  
ALGORITHM = AES_256,  
SERVER CERTIFICATE = TDECert  
);  
GO
```

В результате в папке должны появиться бэкап и сертификаты

Компьютер > Локальный диск (C:) > SQLBackups



3.3 Разработка политики безопасности SQL сервера

В ходе данного задания была разработана политика безопасности SQL сервера. Процесс создания включал в себя анализ документации и интернет-ресурсов на эту тему, а итогом работы стал структурированный документ, описывающий основные и наиболее эффективные средства обеспечения безопасности SQL-сервера.

(Скриншот политики)

В ходе выполнения практики были реализованы меры, по обеспечению данной политики. Например, в целях резервного копирования была создана зашифрованная резервная копия базы данных с использованием встроенных средств SQL Server.

Для этого были выполнены следующие этапы:

1. Создан мастер-ключ в базе данных master, используемый для защиты сертификатов.
2. Создан серверный сертификат TDECert, применяемый для шифрования резервных копий.
3. Выполнена проверка наличия сертификата.
4. Создана резервная копия сертификата и закрытого ключа в отдельной директории.
5. Выполнено резервное копирование базы данных с использованием алгоритма AES-256.

(скриншоты запросов) (Крин папки бэкапа)

Практическая реализация показала, что большинство требований политики безопасности могут быть непосредственно реализованы средствами СУБД, что позволяет эффективно защищать данные и контролировать доступ пользователей.